

FBG 센서 데이터 분석 코드 정리

연구과제명	광융합 기반 콘크리트 구조물 시공 단계 양생품질 계측관리 기술 개발
연구기간	2025. 1. 1. ~ 2025. 12. 31.
연구책임자	박형준
기록자	김영환
확인자	김대길
작성일자	2025.8.25



1. 기본 사항

날짜: 2025-07-22 ~ 2025-08-19

장소: 2층, 광ICT융합연구실

2. 목적

FBG(Fiber Bragg Grating) 센서로부터 수집된 데이터는 라인별로 센서 개수와 배치가 다르고, 실험 도중 특정 시점 이후 일부 센서가 비활성화되거나 결측 구간이 발생하는 등 데이터 형식이 일정하지 않다. 이러한 특성 때문에 분석 전처리 과정에서 불필요한 오류가 발생하기 쉽고, 단순 수작업 처리로는 데이터의 정확성과 일관성을 유지하기 어렵다. 따라서 본 보고서는 데이터 구조의 불규칙성을 정리하고, 반복적으로 등장하는 전처리 과정을 코드로 통합·자동화함으로써 데이터 품질 관리와 분석 효율성을 높이고자 한다.

3. 사용 장비

파이썬

4. 내용

1) get_file_paths

목적:

이 함수는 FBG(A~E) 실험 데이터가 날짜별로 폴더(2025_07_22, 2025_07_23 등)에 저장되어 있을 때,

특정 기간(start_date ~ end_date)의 모든 데이터 파일 경로를 자동으로 수집하기 위해 작성되었다.

```
import pandas as pd
import numpy as np
import glob
import os, re
from datetime import datetime, timedelta

def get_file_paths(root_folder, start_date, end_date, ext="*.txt"):
    """
    root_folder: FBG-A, FBG-B 등 mockup 폴더 경로
    start_date, end_date: 'YYYY-MM-DD' 문자열
    ext: 확장자 (기본 *.txt)
    """
    # 날짜 범위 생성
    start_dt = datetime.strptime(start_date, "%Y-%m-%d")
    end_dt = datetime.strptime(end_date, "%Y-%m-%d")
    days = (end_dt - start_dt).days + 1
```

```
folder_paths = [  
    os.path.join(root_folder, (start_dt +  
timedelta(days=i)).strftime("%Y_%m_%d"))  
    for i in range(days)  
]  
  
# 파일 경로 모으기  
file_paths = []  
for folder in folder_paths:  
    file_paths.extend(glob.glob(os.path.join(folder, ext)))  
  
print(f"[{root_folder}] 총 {len(file_paths)}개 {ext} 파일 불러옴")  
return file_paths
```

동작 원리 :

1. 날짜 범위 지정

- 사용자가 start_date, end_date를 입력하면, 해당 기간 동안의 날짜 리스트를 생성한다.
- 예: 2025-07-22 ~ 2025-07-25 → [2025_07_22, 2025_07_23, 2025_07_24, 2025_07_25]

2. 폴더 경로 생성

- 각 날짜별 폴더명을 루트 경로(root_folder)와 합쳐 실제 데이터가 저장된 디렉터리 경로를 만든다.
- 예: root_folder="C:\FBG-A-mockup" →
C:\FBG-A-mockup\2025_07_22
C:\FBG-A-mockup\2025_07_23
...

3. 파일 검색

- 각 날짜 폴더 내부에서 지정된 확장자(ext, 기본값 "*.txt")를 가진 모든 파일을 탐색한다.
- glob.glob()을 사용하여 파일 리스트를 가져오고, 최종적으로 모든 날짜의 파일들을 하나의 리스트로 합친다.

4. 결과 반환 및 로그 출력

- 파일 경로 리스트(file_paths)를 반환한다.
- 동시에, 해당 root 폴더에서 몇 개의 파일이 수집되었는지 출력한다

활용 예시

```
#files_A = get_file_paths(r"C:\Users\user\Desktop\FBG-A-mockup",  
"2025-07-22", "2025-08-19")  
#files_B = get_file_paths(r"C:\Users\user\Desktop\FBG-B-mockup-strain",  
"2025-07-22", "2025-08-19")
```

```
#files_C =  
get_file_paths(r"C:\Users\user\Desktop\FBG-C-specimen-standard",  
"2025-07-22", "2025-08-19")  
files_D = get_file_paths(r"C:\Users\user\Desktop\FBG-D-specimen",  
"2025-07-22", "2025-08-19")  
#files_E = get_file_paths(r"C:\Users\user\Desktop\FBG-E-mockup-center",  
"2025-07-22", "2025-07-29")  
# 일부 확인  
for path in files_D[:3]:  
    print("D:", path)
```

D의 2025년 7월 22일부터 2025년 8월 19일까지 데이터 파일 경로를 불러옴.

2) process_fbg_line

목적 :

process_fbg_line() 함수는 FBG 센서 라인(A~E) 에 대해,

날짜별 텍스트 파일(*.txt)에 저장된 원시 데이터를 통합적으로 처리하고,

시간 동기화·결측 처리·전환 시점 규칙 적용 후 7일 단위 CSV로 자동 저장하기 위해 작성되었다.

```
import os, re  
import pandas as pd  
import numpy as np  
def process_fbg_line(line_name, file_paths, base_dir, prefix, rules):  
    """  
    FBG 라인(A~E) 통합 처리 함수 (multi_cut 지원)  
    Parameters  
    -----  
    line_name : str  
        "A", "B", "C", "D", "E"  
    file_paths : list  
        *.txt 파일 전체 경로 리스트  
    base_dir : str  
        CSV 저장할 기본 폴더  
    prefix : str  
        출력 파일 접두어  
    rules : dict  
        {  
            "base_cols_main": [...],          # 기본 센서 열 정의  
            "alt_cols": {열개수: [...], ...}, # 열 개수별 정의
```

```
"cut_time": pd.Timestamp(...), # (단일 규칙용) 라인 전환 시점
"shift_rule": ("C_FBG6", "right"), # 시프트 규칙
"drop_after": ['SC_FBG1', ...], # cut_time 이후 비울 센서
"multi_cut": [ # 여러 개의 전환 규칙
    {"type": "shift"/"nullify", "start": pd.Timestamp(...),
"cols": [...]}],
    ...
]
}
"""

# =====
# 기본 준비
# =====
time_pat = re.compile(r'^\d{2}[_:]\d{2}[_:]\d{2}$')
base_cols = rules["base_cols_main"]
sensor_cols = [c for c in base_cols if c != 'Time']
out_cols = ['Time'] + sensor_cols + ['Timestamp']
dfs = []

# =====
# 1) 줄 단위 파싱
# =====
for path in file_paths:
    folder_name = os.path.basename(os.path.dirname(path))
    date_str = folder_name.replace('_', '-')
    try:
        with open(path, encoding='utf-8') as f:
            raw_lines = [ln.strip() for ln in f if ln.strip()]
    except Exception as e:
        print(f"[WARN] 파일 열기 실패: {path} ({e})")
        continue
    records, bad_rows = [], 0
    for tokens in (ln.split() for ln in raw_lines):
        n = len(tokens)
        if n & 1 or not time_pat.match(tokens[0]):
            bad_rows += 1
            continue
        rec = {c: None for c in base_cols}
        rec['Time'] = tokens[0].replace(':', '_')
        if n == len(base_cols):
```

```
        for col, val in zip(base_cols, tokens):
            if col != 'Time':
                rec[col] = val
        elif "alt_cols" in rules and n in rules["alt_cols"]:
            alt = rules["alt_cols"][n]
            for col, val in zip(alt, tokens):
                if col != 'Time':
                    rec[col] = val
        else:
            bad_rows += 1
        records.append(rec)
    if not records:
        print(f"[WARN] {path}: 유효한 데이터 없음")
        continue
    df = pd.DataFrame.from_records(records, columns=base_cols)
    df['Timestamp'] = pd.to_datetime(
        date_str + ' ' + df['Time'].str.replace('_', ':'),
        format='%Y-%m-%d %H:%M:%S',
        errors='coerce'
    )
    df = df.dropna(subset=['Timestamp'])
    # =====
    # 2) 규칙 적용 (cut_time / multi_cut)
    # =====
    if not df.empty:
        # 단일 cut_time 처리
        if "cut_time" in rules:
            cut_time = rules["cut_time"]
            if "shift_rule" in rules:
                start_col, direction = rules["shift_rule"]
                mask_shift = df['Timestamp'] &= cut_time
                if mask_shift.any():
                    cols = [c for c in sensor_cols if c &= start_col]
                    blk = df.loc[mask_shift, cols].copy()
                    if direction == "right":
                        blk.iloc[:, 1:] = blk.iloc[:, :-1].to_numpy()
                        blk.iloc[:, 0] = None
                    df.loc[mask_shift, cols] = blk
            if "drop_after" in rules:
```

```

        mask_after = df['Timestamp'] &= cut_time
        if mask_after.any():
            df.loc[mask_after, rules["drop_after"]] = None
# 다중 cut_time 처리
if "multi_cut" in rules:
    for rule in rules["multi_cut"]:
        rtype = rule["type"]
        rstart = rule["start"]
        rcols = rule["cols"]
        mask = df['Timestamp'] &= rstart
        if rtype == "nullify":
            if mask.any():
                df.loc[mask, rcols] = None
        elif rtype == "shift":
            if mask.any():
                blk = df.loc[mask, rcols].copy()
                blk.iloc[:, 1:] = blk.iloc[:, :-1].to_numpy()
                blk.iloc[:, 0] = None
                df.loc[mask, rcols] = blk

if bad_rows & 0:
    print(f"[INFO] {os.path.basename(path)}: 비정상 줄 {bad_rows}개
")

dfs.append(df)

# =====
# 3) 병합 & 정리
# =====
if not dfs:
    raise RuntimeError(f"[{line_name}] 파싱된 데이터 없음")
merged_df = pd.concat(dfs, ignore_index=True)
merged_df = merged_df.sort_values('Timestamp')
merged_df = merged_df.drop_duplicates(subset=['Timestamp'],
keep='first').reset_index(drop=True)
start_all = merged_df['Timestamp'].min().floor('S')
end_all = merged_df['Timestamp'].max()
print(f"[{line_name}] 전체 구간: {start_all} ~ {end_all} (행
{len(merged_df):,})")

# =====
# 4) 7일 단위 저장 (+1s 타임라인 보강)
# =====

```

```

curr = start_all
part = 1
while curr <= end_all:
    nxt = min(curr + pd.Timedelta(days=7), end_all +
pd.Timedelta(seconds=1))
    seg = merged_df[(merged_df['Timestamp'] <= curr) &
(merged_df['Timestamp'] < nxt)].copy()
    if seg.empty:
        print(f"[{line_name}] [{part}] {curr:%Y-%m-%d} ~
{((nxt-pd.Timedelta(seconds=1)):%Y-%m-%d)}: 데이터 없음")
        curr = nxt; part += 1
        continue
    seg = seg.groupby('Timestamp', as_index=True).first().sort_index()
    full_index = pd.date_range(curr, nxt - pd.Timedelta(seconds=1),
freq='S')
    seg = seg.reindex(full_index)
    seg.index.name = 'Timestamp'
    seg['Time'] = seg.index.strftime('%H_%M_%S')
    for c in sensor_cols:
        if c not in seg.columns:
            seg[c] = pd.NA
    out = seg.reset_index()[out_cols].fillna('')
    fname =
fr"{base_dir}\{prefix}_{curr:%Y%m%d}_{((nxt-pd.Timedelta(seconds=1)):%Y%m%
d)}.csv"
    out.to_csv(fname, index=False, encoding='utf-8-sig')
    print(f"[{line_name}] [SAVE] {fname} ({len(out):,} rows)")
    curr = nxt; part += 1

```

동작 원리 :

1. 기본 준비

- 센서 라인별 규칙(rules)을 입력받아 표준 열(base_cols_main)을 정의한다.
- 파일을 한 줄씩 읽어 시간 포맷(HH_MM_SS)을 검사하고, 센서 값들을 파싱한다.
- 열 개수가 달라질 수 있는 경우(alt_cols)를 허용하여 다양한 데이터 포맷에 대응한다.

2. 라인 전환 시점 처리 (cut_time)

- 실험 중 특정 시점 이후 일부 센서 구조가 변하거나 데이터가 누락되는 경우가 있어,
- shift_rule(오른쪽/왼쪽 시프트), drop_after(특정 센서 무효화) 규칙을 적용한다.

3. 데이터 병합 및 정리

- 파일별 DataFrame을 합쳐 Timestamp 기준으로 정렬한다.
- 중복 Timestamp는 첫 번째 값만 남기고 제거한다.

4. 7일 단위 저장 + 초 단위 보강

- 전체 구간을 7일씩 나눠 잘라 CSV 파일로 저장한다.
- 초 단위 타임라인(1s)으로 보강하여 데이터가 없는 시각은 빈 칸으로 채운다.
- 센서 컬럼 누락 시 자동으로 생성하여 일관된 데이터 구조를 유지한다.

★ 규칙 정의

```
# FBG-A
rules_A = {
    "base_cols_main": [
        'Time', 'C_FBG15', 'C_FBG14', 'C_FBG13', 'C_FBG12', 'C_FBG11',
        'C_FBG10', 'C_FBG9', 'C_FBG8', 'C_FBG7', 'C_FBG6'
    ],
    "alt_cols": {
        11: ['Time', 'C_FBG15', 'C_FBG14', 'C_FBG13', 'C_FBG12', 'C_FBG11',
            'C_FBG10', 'C_FBG9', 'C_FBG8', 'C_FBG7', 'C_FBG6'],
        10: ['Time', 'C_FBG14', 'C_FBG13', 'C_FBG12', 'C_FBG11',
            'C_FBG10', 'C_FBG9', 'C_FBG8', 'C_FBG7', 'C_FBG6'], # C_FBG15는
None
    },
    "multi_cut": [
        {"type": "shift", "start": pd.Timestamp("2025-07-25 11:09:03"),
         "cols": ['C_FBG15', 'C_FBG14', 'C_FBG13', 'C_FBG12', 'C_FBG11',
                  'C_FBG10', 'C_FBG9', 'C_FBG8', 'C_FBG7', 'C_FBG6']}
    ]
}

# FBG-B
rules_B = {
    "base_cols_main": [
        'Time', 'C_FBG5', 'C_FBG4', 'C_FBG3', 'C_FBG2', 'C_FBG1',
        'CS_FBG3', 'CS_FBG2', 'CS_FBG1'
    ],
    "alt_cols": {
        9: ['Time', 'C_FBG5', 'C_FBG4', 'C_FBG3', 'C_FBG2', 'C_FBG1',
            'CS_FBG3', 'CS_FBG2', 'CS_FBG1'],
    }
```

```
8: ['Time', 'C_FBG5', 'C_FBG4', 'C_FBG3', 'C_FBG2', 'C_FBG1',
    'CS_FBG3', 'CS_FBG2'], # CS_FBG1은 None
},
"multi_cut": [
    {"type": "nullify", "start": pd.Timestamp("2025-07-29 10:01:25"),
     "cols": ['CS_FBG1']}
]
}

# FBG-C
rules_C = {
    "base_cols_main": [
        'Time', 'SC_FBG1', 'SC_FBG2', 'SC_FBG3', 'SC_FBG4', 'SC_FBG5',
        'SC_FBG6', 'SC_FBG7', 'SC_FBG8', 'SC_FBG9', 'SC_FBG10', 'SC_FBG11'
    ],
    "alt_cols": {
        12: ['Time', 'SC_FBG1', 'SC_FBG2', 'SC_FBG3', 'SC_FBG4', 'SC_FBG5',
            'SC_FBG6', 'SC_FBG7', 'SC_FBG8', 'SC_FBG9', 'SC_FBG10', 'SC_FBG11'],
        9: ['Time', 'SC_FBG3', 'SC_FBG4', 'SC_FBG6', 'SC_FBG7', 'SC_FBG8',
            'SC_FBG9', 'SC_FBG10', 'SC_FBG11'],
        6: ['Time', 'SC_FBG7', 'SC_FBG8', 'SC_FBG9', 'SC_FBG10', 'SC_FBG11'],
    },
    "multi_cut": [
        {"type": "nullify", "start": pd.Timestamp("2025-07-25 09:58:50"),
         "cols": ['SC_FBG1', 'SC_FBG2', 'SC_FBG5']},
        {"type": "nullify", "start": pd.Timestamp("2025-07-29 09:55:15"),
         "cols": ['SC_FBG3', 'SC_FBG4', 'SC_FBG6']}
    ]
}

# FBG-D
rules_D = {
    "base_cols_main": [
        'Time', 'AD_FBG1', 'AD_FBG2', 'AD_FBG3', 'AD_FBG4', 'AD_FBG5',
        'AD_FBG6', 'AD_FBG7', 'AD_FBG8', 'AD_FBG9', 'AD_FBG10', 'AD_FBG11'
    ],
    "alt_cols": {
        12: ['Time', 'AD_FBG1', 'AD_FBG2', 'AD_FBG3', 'AD_FBG4', 'AD_FBG5',
```

```

'AD_FBG6','AD_FBG7','AD_FBG8','AD_FBG9','AD_FBG10','AD_FBG11'],
    9: ['Time','AD_FBG3','AD_FBG4','AD_FBG6','AD_FBG7','AD_FBG8',
        'AD_FBG9','AD_FBG10','AD_FBG11'],
    6: ['Time','AD_FBG6','AD_FBG7','AD_FBG9','AD_FBG10','AD_FBG11'],
},
"multi_cut": [
    {"type": "nullify", "start": pd.Timestamp("2025-07-25 10:00:29"),
      "cols": ['AD_FBG1','AD_FBG2','AD_FBG5']},
    {"type": "nullify", "start": pd.Timestamp("2025-07-29 09:52:30"),
      "cols": ['AD_FBG3','AD_FBG4','AD_FBG8']}
]
}

# FBG-E
rules_E = {
    "base_cols_main": ['Time','CC_FBG1','CC_FBG2','CC_FBG3'],
    "alt_cols": {},
    "multi_cut": []
}

```

FBG 라인별 Rules 설명

1. FBG-A

- 데이터 특징
 - 기본 구조: 11열 (Time + C_FBG15 ~ C_FBG6)
 - 경우에 따라 10열만 기록되는 케이스 존재 (C_FBG15 빠짐)
 - 2025-07-25 11:09:03 이후 → 센서 값이 한 칸 오른쪽으로 shift
- 규칙 정의
 - "base_cols_main" : Time + C_FBG15 ~ C_FBG6 (정상 구조)
 - "alt_cols" : 열이 10개면 C_FBG15는 None으로 채움
 - "cut_time" : 2025-07-25 11:09:03
 - "shift_rule" : C_FBG15부터 오른쪽으로 한 칸 이동

2. FBG-B

- 데이터 특징
 - 기본 구조: 9열 (Time + C_FBG5~1, CS_FBG3~1)
 - 경우에 따라 8열만 기록되는 케이스 존재 (CS_FBG1 빠짐)
 - 2025-07-29 10:01:25 이후 → CS_FBG1은 데이터가 존재하지 않음
- 규칙 정의

- "base_cols_main" : Time + C_FBG5~1 + CS_FBG3~1
- "alt_cols" : 열이 8개면 CS_FBG1은 None 처리
- "cut_time" : 2025-07-29 10:01:25
- "drop_after" : CS_FBG1을 Null로 설정

3. FBG-C

- 데이터 특징
 - 기본 구조: 12열 (Time + SC_FBG1~11)
 - 경우에 따라 9열(1,2,5 없음) / 6열(1~6 없음) 기록
 - 두 번의 라인 전환 발생:
 - 2025-07-25 09:58:50 → SC_FBG1,2,5 없음
 - 2025-07-29 09:55:15 → SC_FBG3,4,6 없음
- 규칙 정의
 - "base_cols_main" : Time + SC_FBG1~11
 - "alt_cols" : 9열/6열 케이스 정의
 - "multi_cut" :
 - (1) cut_time1(2025-07-25 09:58:50) → SC_FBG1,2,5 drop
 - (2) cut_time2(2025-07-29 09:55:15) → SC_FBG3,4,6 drop

4. FBG-D

- 데이터 특징
 - 기본 구조: 12열 (Time + AD_FBG1~11)
 - 경우에 따라 9열(1,2,5 없음) / 6열(1,2,3,4,5,8 없음) 기록
 - 두 번의 라인 전환 발생:
 - 2025-07-25 10:00:29 → AD_FBG1,2,5 없음
 - 2025-07-29 09:52:30 → AD_FBG3,4,8 없음
- 규칙 정의
 - "base_cols_main" : Time + AD_FBG1~11
 - "alt_cols" : 9열/6열 케이스 정의
 - "multi_cut" :
 - (1) cut_time1(2025-07-25 10:00:29) → AD_FBG3,4,8 drop
 - (2) cut_time2(2025-07-29 09:52:30) → AD_FBG3,4,8 drop

5. FBG-E

- 데이터 특징
 - 항상 4열 (Time + CC_FBG1~3)
 - 특수 규칙 없음
- 규칙 정의
 - "base_cols_main" : Time + CC_FBG1~3
 - "alt_cols" : 없음
 - cut_time / shift / drop_after 모두 필요 없음

활용 예시

```
# FBG-D 처리
process_fbg_line("D", files_D,
                 r"C:\Users\user\Desktop\FBG-D-specimen",
```

```
"merged_FBG-D_1s", rules_D)
```

결과

 merged_FBG-D_1s_20250722_20250729

 merged_FBG-D_1s_20250729_20250805

 merged_FBG-D_1s_20250805_20250812

 merged_FBG-D_1s_20250812_20250819

3) merge_and_resample()

목적 :

- 여러 개의 주차별 센서 데이터(CSV)를 하나의 연속된 시계열 데이터로 통합
- 지정된 주기(기본 2분) 단위로 센서별 평균값을 계산
- 센서 값의 일관성 확보 및 분석·시각화를 위한 전처리 단계 수행

```
import pandas as pd
import numpy as np
def merge_and_resample(paths, sensor_cols, out_path, freq='2T'):
    """
    여러 주차별 merged CSV를 병합하고 센서 컬럼을 기준으로 resample 평균 계산

    Parameters
    -----
    paths : list
        병합할 CSV 파일 경로 리스트
    sensor_cols : list
        센서 컬럼 이름들
    out_path : str
        저장할 CSV 파일 경로
    freq : str
        리샘플링 주기 (기본값 '2T' = 2분)
    """
    dfs = []
    for path in paths:
        df = pd.read_csv(path, parse_dates=['Timestamp'])
        dfs.append(df)
    merged_df = pd.concat(dfs, ignore_index=True)
    merged_df = merged_df.sort_values('Timestamp').set_index('Timestamp')
    # 센서 값 숫자 변환
    for col in sensor_cols:
        merged_df[col] = pd.to_numeric(merged_df[col], errors='coerce')
```

```
# 주기별 평균
df_avg = merged_df[sensor_cols].resample(freq).mean().reset_index()
# 저장
df_avg.to_csv(out_path, index=False, encoding='utf-8-sig')
print(f"    평균 CSV 저장 완료 → {out_path}")
return df_avg
```

동작 과정 :

1. 여러 주차 CSV 파일 로드
 - paths 리스트의 모든 파일을 불러와 DataFrame으로 변환
2. 하나의 DataFrame으로 병합
 - pd.concat()을 이용해 모든 데이터를 연결
 - Timestamp 컬럼 기준으로 정렬하고 인덱스 설정
3. 센서 값 숫자형 변환
 - 문자열/결측치가 섞여 있어도 pd.to_numeric()으로 안전하게 변환
4. 리샘플링(Resample) 평균 계산
 - 주어진 freq 간격으로 센서별 평균값 계산
5. CSV 저장
 - 최종 결과를 out_path에 저장 (UTF-8-SIG 인코딩)

✓ 센서 컬럼 정의

```
sensor_cols_A = [
    'C_FBG15', 'C_FBG14', 'C_FBG13', 'C_FBG12', 'C_FBG11',
    'C_FBG10', 'C_FBG9', 'C_FBG8', 'C_FBG7', 'C_FBG6'
]
sensor_cols_B = [
    'C_FBG5', 'C_FBG4', 'C_FBG3', 'C_FBG2', 'C_FBG1',
    'CS_FBG3', 'CS_FBG2', 'CS_FBG1'
]
sensor_cols_C = [
    'SC_FBG1', 'SC_FBG2', 'SC_FBG3', 'SC_FBG4', 'SC_FBG5',
    'SC_FBG6', 'SC_FBG7', 'SC_FBG8', 'SC_FBG9', 'SC_FBG10', 'SC_FBG11'
]
sensor_cols_D = [
    'AD_FBG1', 'AD_FBG2', 'AD_FBG3', 'AD_FBG4', 'AD_FBG5',
    'AD_FBG6', 'AD_FBG7', 'AD_FBG8', 'AD_FBG9', 'AD_FBG10', 'AD_FBG11'
]
```

```
]
sensor_cols_E = [
    'CC_FBG1', 'CC_FBG2', 'CC_FBG3'
]
```

활용 예시

```
paths_D = [
    r"C:\Users\user\Desktop\FBG-D-specimen\merged_FBG-D_week1.csv",
    r"C:\Users\user\Desktop\FBG-D-specimen\merged_FBG-D_week2.csv",
    r"C:\Users\user\Desktop\FBG-D-specimen\merged_FBG-D_week3.csv",
    r"C:\Users\user\Desktop\FBG-D-specimen\merged_FBG-D_week4.csv",
]
out_D =
r"C:\Users\user\Desktop\FBG-D-specimen\FBG-D_merged_avg_2min_28Days.csv"
merge_and_resample(paths_D, sensor_cols_D, out_D, freq='2T')
```

참고) 이때, “merged_FBG-D_1s”로 되어있던 파일에서 수작업으로 보정한 파일이 “merged_FBG-D_week” 파일임.

결과

 FBG-D_merged_avg_2min_28Days

	A	B	C	D	E	F	G	H	I	J	K	L
1	Timestamp	AD_FBG1	AD_FBG2	AD_FBG3	AD_FBG4	AD_FBG5	AD_FBG6	AD_FBG7	AD_FBG8	AD_FBG9	AD_FBG10	AD_FBG11
2	2025-07-22 11:32	1556.711	1554.654	1550.848	1547.972	1544.933	1543	1540.981	1539.138	1535.98	1533.151	1530.176
3	2025-07-22 11:34	1556.711	1554.654	1550.848	1547.972	1544.933	1542.999	1540.981	1539.138	1535.979	1533.149	1530.171
4	2025-07-22 11:36	1556.711	1554.653	1550.848	1547.972	1544.933	1542.999	1540.981	1539.138	1535.979	1533.149	1530.169
5	2025-07-22 11:38	1556.711	1554.653	1550.848	1547.972	1544.933	1542.999	1540.981	1539.138	1535.979	1533.149	1530.169
6	2025-07-22 11:40	1556.711	1554.653	1550.848	1547.972	1544.933	1542.999	1540.981	1539.138	1535.979	1533.149	1530.167
7	2025-07-22 11:42	1556.711	1554.653	1550.848	1547.972	1544.933	1542.999	1540.981	1539.138	1535.979	1533.149	1530.167
8	2025-07-22 11:44	1556.711	1554.653	1550.848	1547.972	1544.933	1542.999	1540.981	1539.138	1535.979	1533.149	1530.167
9	2025-07-22 11:46	1556.711	1554.653	1550.848	1547.972	1544.933	1542.999	1540.981	1539.137	1535.979	1533.149	1530.168
10	2025-07-22 11:48	1556.711	1554.653	1550.848	1547.972	1544.933	1542.999	1540.981	1539.137	1535.979	1533.149	1530.167
11	2025-07-22 11:50	1556.711	1554.653	1550.848	1547.971	1544.933	1542.999	1540.981	1539.137	1535.979	1533.149	1530.167
12	2025-07-22 11:52	1556.711	1554.653	1550.848	1547.971	1544.933	1542.999	1540.981	1539.137	1535.979	1533.148	1530.168
13	2025-07-22 11:54	1556.711	1554.652	1550.847	1547.971	1544.933	1542.999	1540.98	1539.137	1535.978	1533.148	1530.169

4) convert_wavelength_to_temp()

목적 :

- FBG(Fiber Bragg Grating) 센서에서 측정된 파장(λ)을 “온도($^{\circ}\text{C}$)”로 변환
- 변환된 데이터를 CSV로 저장하여 분석, 시각화 및 보정 단계에서 활용 가능하도록 함
- 파장 데이터와 온도 데이터를 모두 보관하거나, 온도 값만 따로 보관 가능

```
import pandas as pd
import numpy as np
def convert_wavelength_to_temp(input_csv, sensor_cols, prefix="FBG-D"): #
```

prefix 원하는 센서로 설정

```
"""
    FBG 센서 파장을 온도로 변환 후 CSV 저장

    Parameters
    -----
    input_csv : str
        2분 평균된 merged CSV 경로 (예: "FBG-D_merged_avg_2min_28Days.csv")
    sensor_cols : list
        센서 컬럼 이름 리스트 (예: sensor_cols_D)
    prefix : str
        저장할 파일 이름 접두어 (예: "FBG-D")
    """

# 1) 데이터 로드
df = pd.read_csv(input_csv, parse_dates=['Timestamp'])
df = df.sort_values('Timestamp').reset_index(drop=True)

# 2) 센서 값 숫자형 변환
df[sensor_cols] = df[sensor_cols].apply(pd.to_numeric,
errors='coerce')

# 3) 기준 파장  $\lambda_{B0}$  (각 센서 첫 유효값)
lambda_B0 = df[sensor_cols].apply(lambda s:
s.loc[s.first_valid_index()] if s.first_valid_index() is not None else
np.nan)

# 4) 온도 변환 (백터화)
T_0 = 25.0
temp_df = T_0 + (df[sensor_cols] - lambda_B0) / 0.010
temp_df.columns = [c.replace(sensor_cols[0][:6],
f"Temp_{sensor_cols[0][:6]}") for c in sensor_cols]

# 5) 원본 + 온도 열 저장
df_out = pd.concat([df, temp_df], axis=1)
out_path1 = f"{prefix}_temp&wave_avg_2min_28Days.csv"
df_out.to_csv(out_path1, index=False, encoding='utf-8-sig')
print(f"온도 변환 + 파장 CSV 저장 완료 → {out_path1}")

# 6) 온도 값만 저장 (컬럼 이름은 다시 원래 파장 이름으로)
renamed_cols = sensor_cols
df_temp_only = pd.concat([df[['Timestamp']],
temp_df.set_axis(renamed_cols, axis=1)], axis=1)
out_path2 = f"{prefix}_temp_only_avg_2min_28Days.csv"
```



```
df_temp_only.to_csv(out_path2, index=False, encoding='utf-8-sig')
print(f" 온도 전용 CSV 저장 완료 → {out_path2}")
return df_out, df_temp_only
```

동작 과정 :

1. 데이터 로드
 - CSV 불러오기 (Timestamp 파싱, 정렬)
2. 센서 값 숫자 변환
 - 문자열/NaN 값 제거 후 수치형 변환
3. 기준 파장 λ_{B0} 산정
 - 각 센서별 첫 번째 유효값을 기준값 λ_{B0} 로 설정
 - 이후의 값에서 λ_{B0} 를 빼서 변화를 추적
4. 온도 변환
 - 변환 공식: $T = T_0 + (\lambda - \lambda_0)/k$

$$T_0 = 25.0^{\circ}\text{C} \text{ (기준 온도)}$$
$$k = 0.010 \text{ nm}/^{\circ}\text{C} \text{ (온도 민감도 계수)}$$

5. CSV 저장 (2종류)
 - (1) 원본 파장 + 변환된 온도 함께 저장
→ "FBG-D_temp&wave_avg_2min_28Days.csv"
 - (2) 온도 값만 저장 (컬럼명은 원래 파장 이름 유지)
→ "FBG-D_temp_only_avg_2min_28Days.csv"

활용 예시

```
# D라인 변환 실행
convert_wavelength_to_temp(
    input_csv=r"C:\Users\user\Desktop\FBG-D-specimen\FBG-D_merged_avg_2min_28Days.csv",
    sensor_cols=sensor_cols_D,
    prefix=r"C:\Users\user\Desktop\FBG-D-specimen\FBG-D"
)
```

결과

 FBG-D_temp&wave_avg_2min_28Days

 FBG-D_temp_only_avg_2min_28Days

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
1	Timestamp	AD_FBG1	AD_FBG2	AD_FBG3	AD_FBG4	AD_FBG5	AD_FBG6	AD_FBG7	AD_FBG8	AD_FBG9	AD_FBG10	AD_FBG11	Temp_AD_1	Temp_AD_2	Temp_AD_3	Temp_AD_4	Temp_AD_5	Temp_AD_6
2	2025-07-22 11:32	1556.711	1554.654	1550.848	1547.972	1544.933	1543	1540.981	1539.138	1535.98	1533.151	1530.176	25	25	25	25	25	25
3	2025-07-22 11:34	1556.711	1554.654	1550.848	1547.972	1544.933	1542.999	1540.981	1539.138	1535.979	1533.149	1530.171	25.00178	24.99477	24.99313	24.98846	24.99221	24.99249
4	2025-07-22 11:36	1556.711	1554.653	1550.848	1547.972	1544.933	1542.999	1540.981	1539.138	1535.979	1533.149	1530.169	25.00487	24.98777	24.98621	24.98596	24.97812	24.99649
5	2025-07-22 11:38	1556.711	1554.653	1550.848	1547.972	1544.933	1542.999	1540.981	1539.138	1535.979	1533.149	1530.169	25.00512	24.97861	24.98205	24.98013	24.98646	24.98199
6	2025-07-22 11:40	1556.711	1554.653	1550.848	1547.972	1544.933	1542.999	1540.981	1539.138	1535.979	1533.149	1530.167	25.00295	24.97452	24.98371	24.96846	24.97921	24.96958
7	2025-07-22 11:42	1556.711	1554.653	1550.848	1547.972	1544.933	1542.999	1540.981	1539.138	1535.979	1533.149	1530.167	25.00153	24.96002	24.97513	24.96429	24.96679	24.95491
8	2025-07-22 11:44	1556.711	1554.653	1550.848	1547.972	1544.933	1542.999	1540.981	1539.138	1535.979	1533.149	1530.167	25.00112	24.95119	24.9703	24.95788	24.95521	24.93949
9	2025-07-22 11:46	1556.711	1554.653	1550.848	1547.972	1544.933	1542.999	1540.981	1539.137	1535.979	1533.149	1530.168	25.00412	24.93994	24.95963	24.95063	24.96171	24.94233
10	2025-07-22 11:48	1556.711	1554.653	1550.848	1547.972	1544.933	1542.999	1540.981	1539.137	1535.979	1533.149	1530.167	25.00237	24.92661	24.95846	24.94321	24.94312	24.93049
	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
1	Timestamp	AD_FBG1	AD_FBG2	AD_FBG3	AD_FBG4	AD_FBG5	AD_FBG6	AD_FBG7	AD_FBG8	AD_FBG9	AD_FBG10	AD_FBG11						
2	2025-07-22 11:32	25	25	25	25	25	25	25	25	25	25	25						
3	2025-07-22 11:34	25.00178	24.99477	24.99313	24.98846	24.99221	24.99249	24.99746	25.00077	24.99025	24.87749	24.51447						
4	2025-07-22 11:36	25.00487	24.98777	24.98621	24.98596	24.97812	24.99649	24.99579	24.99752	24.98991	24.84882	24.34272						
5	2025-07-22 11:38	25.00512	24.97861	24.98205	24.98013	24.98646	24.98199	24.98371	24.98869	24.97916	24.85641	24.28206						
6	2025-07-22 11:40	25.00295	24.97452	24.98371	24.96846	24.97921	24.96958	24.98196	24.99227	24.97341	24.84882	24.14906						
7	2025-07-22 11:42	25.00153	24.96002	24.97513	24.96429	24.96679	24.95491	24.97663	24.97977	24.96366	24.83174	24.11722						
8	2025-07-22 11:44	25.00112	24.95119	24.9703	24.95788	24.95521	24.93949	24.96721	24.97636	24.95058	24.82624	24.10297						
9	2025-07-22 11:46	25.00412	24.93994	24.95963	24.95063	24.96171	24.94233	24.95671	24.96369	24.93758	24.82582	24.20481						
10	2025-07-22 11:48	25.00237	24.92661	24.95846	24.94321	24.94312	24.93049	24.95088	24.96327	24.92908	24.80274	24.08231						
11	2025-07-22 11:50	25.00178	24.91469	24.9473	24.93546	24.95712	24.91024	24.93471	24.94969	24.92616	24.79332	24.13031						

5) plot_fbg_temps()

목적 :

- FBG 센서의 온도 변환 값을 시간 축으로 개별 플롯
- 초기 파장 λ_{B0} 값을 그래프에 주석으로 표시
- 결측 구간(NaN 연속 구간) 은 시각적으로 음영 처리
- 원하는 y축 범위 직접 지정 가능

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
from matplotlib.ticker import FormatStrFormatter

def plot_fbg_temps(temp_csv, wave_csv, sensor_cols,
                   start_time, end_time, y_limits=None, prefix="FBG-D"):
    """
    FBG 센서(온도 변환값 + 초기 파장) 개별 플롯

    Parameters
    -----
    temp_csv : str
        온도 CSV (예: "FBG-D_temp_only_avg_2min_28Days.csv")
    wave_csv : str
        원래 파장 CSV (예: "FBG-D_merged_avg_2min_28Days.csv")
    sensor_cols : list
```

```
    센서 이름 리스트 (예: sensor_cols_D)
    start_time : pd.Timestamp
        시작 시각
    end_time : pd.Timestamp
        끝 시각
    y_limits : tuple (ymin, ymax), optional
        y축 범위를 직접 지정할 때 사용. None이면 데이터 기반 자동 계산.
    prefix : str
        그래프 제목에 들어갈 접두어
    """
    # 1) 데이터 로드
    avg_df = pd.read_csv(temp_csv,
        parse_dates=['Timestamp']).sort_values('Timestamp').reset_index(drop=True)

    df_wave = pd.read_csv(wave_csv, parse_dates=['Timestamp'])

    # 2) 구간 선택
    window = avg_df[(avg_df['Timestamp'] >= start_time) &
        (avg_df['Timestamp'] <= end_time)]

    # 3) 누락 구간 탐지
    bin_sec = 120
    miss_mask = avg_df[sensor_cols].isna().all(axis=1).values
    ts = avg_df['Timestamp'].values
    gap_ranges = []
    if len(miss_mask):
        padded = np.r_[False, miss_mask, False]
        edges = np.flatnonzero(padded[1:] != padded[:-1])
        for a, b in zip(edges[0::2], edges[1::2]):
            dur_sec = (b - a) * bin_sec
            if dur_sec >= 120:
                st = pd.Timestamp(ts[a])
                en = pd.Timestamp(ts[b-1]) + pd.Timedelta(seconds=bin_sec)
                gap_ranges.append((st, en))

    # 4) y축 범위
    all_vals = window[sensor_cols].stack().astype(float)
    global_min, global_max = all_vals.min(), all_vals.max()
    if y_limits is None:
        y_margin = (global_max - global_min) * 0.1 if
```

```
np.isfinite(global_min) and np.isfinite(global_max) else 1.0
    y_lo, y_hi = global_min - y_margin, global_max + y_margin
else:
    y_lo, y_hi = y_limits
# 5) 초기 파장  $\lambda_{B0}$ 
    lambda_B0 = df_wave.loc[0, sensor_cols] if
set(sensor_cols).issubset(df_wave.columns) else pd.Series(dtype=float)
# 6) 센서별 플롯
for col in sensor_cols:
    temp_df = window[['Timestamp', col]].copy()
    temp_df[col] = pd.to_numeric(temp_df[col], errors='coerce')
    # 누락구간 NaN 처리
    for st, en in gap_ranges:
        if st & end_time and en & start_time:
            m = (temp_df['Timestamp'] &= max(st, start_time)) &
(temp_df['Timestamp'] & min(en, end_time))
            temp_df.loc[m, col] = np.nan
    y = temp_df[col].astype(float)
    fig, ax = plt.subplots(figsize=(12, 4))
    # 누락 음영
    for st, en in gap_ranges:
        if st & end_time and en & start_time:
            ax.axvspan(max(st, start_time), min(en, end_time),
color='skyblue', alpha=0.35, zorder=0)
    # 데이터
    ax.scatter(temp_df['Timestamp'], y, label=col, s=5, zorder=5)
    # 초기 파장 주석
    if col in lambda_B0.index:
        init_lambda = float(lambda_B0[col])
        y_for_note = (y.max() if np.isfinite(y.max()) else 0.0)
        ax.annotate(f"Init  $\lambda$ : {init_lambda:.2f} nm",
                    xy=(start_time, y_for_note), xytext=(0, 10),
                    textcoords='offset points', fontsize=9,
color='red')
    # 포매팅
    ax.set_title(f'{prefix} - {col} vs Time (avg_2min)')
    ax.set_xlabel('Time')
    ax.set_ylabel('Temperature (°C)')
    ax.grid(True)
```

```
ax.legend(loc='upper right')
ax.set_xlim([start_time, end_time])
ax.set_ylim([y_lo, y_hi])
ax.yaxis.set_major_formatter(FormatStrFormatter('%.2f'))
ax.xaxis.set_major_formatter(mdates.DateFormatter('%m-%d %H:%M'))
ax.xaxis.set_major_locator(mdates.HourLocator(interval=24))
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

동작 과정 :

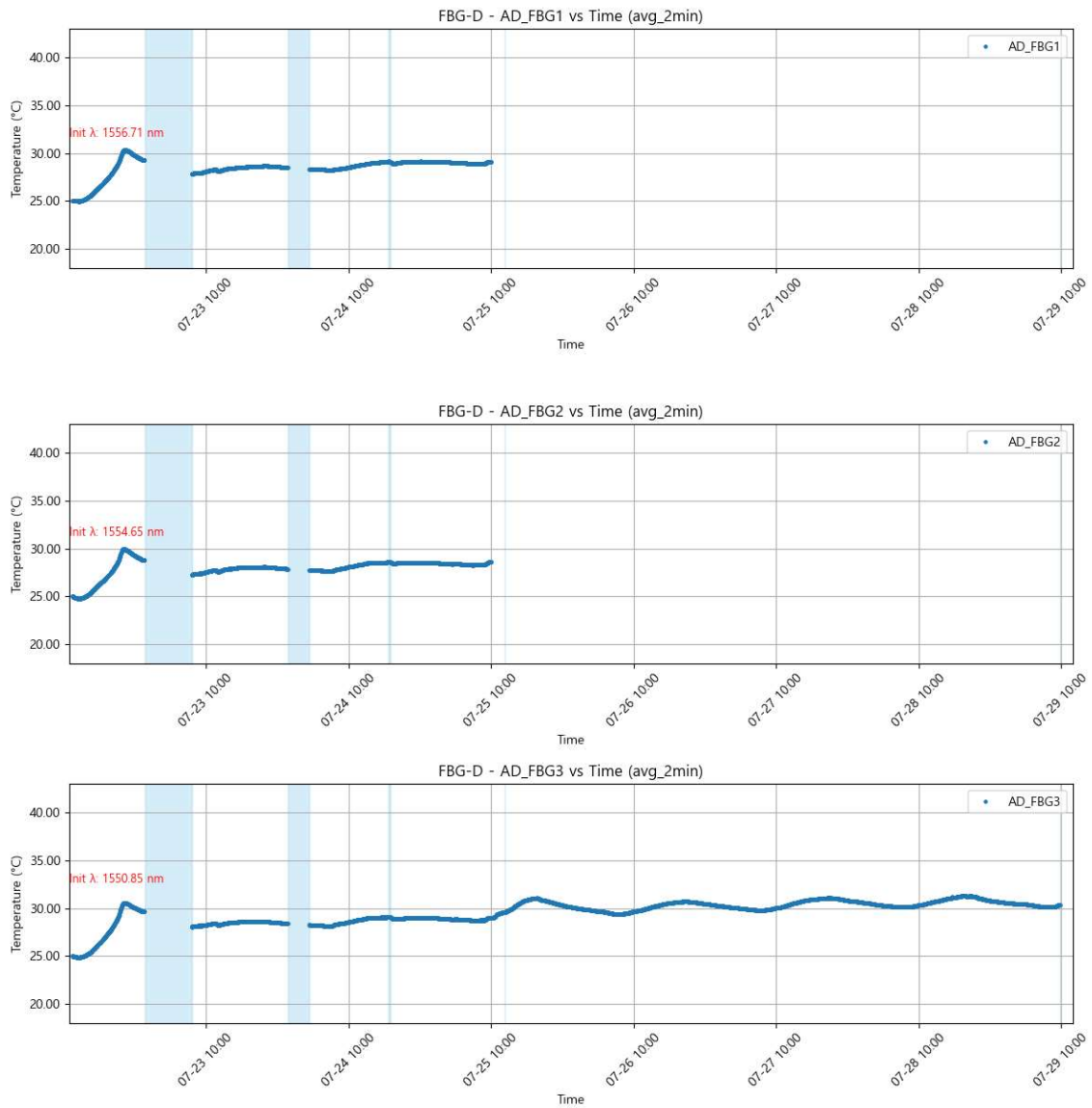
1. 데이터 로드 및 정렬
 - 온도 CSV, 파장 CSV 불러오기
2. 관심 구간(window) 선택
 - start_time ~ end_time 사이 데이터만 사용
3. 누락 구간 탐지
 - 모든 센서 값이 NaN인 연속 구간을 찾아 음영 처리
4. y축 범위 설정
 - 지정값(y_limits)이 없으면 자동 계산
5. 초기 파장 λ_{B0} 산정
 - 파장 CSV에서 첫 번째 시점의 값을 초기 파장으로 설정
6. 센서별 개별 플롯 생성
 - 온도 데이터를 scatter로 플로팅
 - 누락 구간은 빈칸 처리 + 음영 표시
 - 그래프 상단에 초기 파장(λ_{B0}) 값 주석 표시

활용 예시

```
plot_fbg_temps(
    temp_csv=r"C:\Users\user\Desktop\FBG-D-specimen\FBG-D_temp_only_avg_2min_28Days.csv",
    wave_csv=r"C:\Users\user\Desktop\FBG-D-specimen\FBG-D_merged_avg_2min_28Days.csv",
    sensor_cols=sensor_cols_D,
    start_time=pd.Timestamp('2025-07-22 11:00:00'),
    end_time=pd.Timestamp('2025-07-29 12:00:00'),
```

```
y_limits=(18, 43), # 직접 y축 범위 설정 가능  
prefix="FBG-D"  
)
```

결과



6) plot_fbg_vs_iot()

목적 :

- “IoT 데이터(온도, 습도)”와 “FBG 센서 데이터(온도 변환값)”를 같은 시간축에서 비교 시각화
- 분석자가 원하는 센서만 선택 가능
- IoT 온도·습도는 옵션으로 표시/숨기기 가능

- 결측 구간을 자동 탐지해서 음영 표시
- 특정 이벤트 시각(mark_times)을 빨간 점선으로 표시

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
from matplotlib.ticker import FormatStrFormatter

def plot_fbg_vs_iot(
    iot_csv, fbg_csv, sensor_prefix,
    start_time="2025-07-22 11:00:00",
    end_time="2025-08-19 12:00:00",
    y_limits=(18, 43),
    mark_times=None,
    selected_sensors=None,
    show_iot_temp=True,
    show_iot_humidity=True,
    title="FBG vs IoT Temp & Humidity"
):
    """
    IoT 데이터와 FBG 센서 데이터를 비교 플로팅하는 함수

    Parameters
    -----
    iot_csv : str
        IoT CSV 파일 경로
    fbg_csv : str
        FBG 온도 CSV 파일 경로
    sensor_prefix : str
        센서 컬럼 접두어 (예: "AD_FBG", "C_FBG", "SC_FBG", "CC_FBG")
    start_time : str
        그래프 시작 시각
    end_time : str
        그래프 종료 시각
    y_limits : tuple
        (ymin, ymax) 수동 설정 (None이면 자동)
    mark_times : list
        특정 시각에 빨간 점선 표시 (리스트)
    selected_sensors : list
        특정 센서만 선택 (예: ["AD_FBG9", "AD_FBG10"])
```

```
show_iot_temp : bool
    IoT 온도 표시 여부
show_iot_humidity : bool
    IoT 습도 표시 여부
title : str
    그래프 제목
"""

# ----- IoT CSV 로드 -----
iot = pd.read_csv(iot_csv)
iot = iot.rename(columns={
    'Timestamp': 'datetime',
    'Temperature(C)': 'Temperature',
    'Temperatu': 'Temperature',
    'Humidity(%)': 'Humidity',
    'humidity': 'Humidity'
})
iot['datetime'] = pd.to_datetime(iot['datetime'], errors='coerce')
iot['Temperature'] = pd.to_numeric(iot['Temperature'], errors='coerce')
iot['Humidity'] = pd.to_numeric(iot['Humidity'], errors='coerce')

# ----- FBG CSV 로드 -----
avg_df = pd.read_csv(fbg_csv,
    parse_dates=['Timestamp']).sort_values('Timestamp').reset_index(drop=True)

temp_cols = [c for c in avg_df.columns if c.startswith(sensor_prefix)]
# 특정 센서만 선택
if selected_sensors:
    temp_cols = [c for c in temp_cols if c in selected_sensors]

# ----- 시간 범위 -----
start_time = pd.Timestamp(start_time)
end_time = pd.Timestamp(end_time)
win_fbg = avg_df[(avg_df['Timestamp'] >= start_time) &
    (avg_df['Timestamp'] < end_time)]
win_iot = iot[(iot['datetime'] >= start_time) & (iot['datetime'] <
    end_time)].copy()

# ----- 누락구간 탐지 -----
bin_sec = 120
miss_mask = avg_df[temp_cols].isna().all(axis=1).values if temp_cols
else []
```



```
ts = avg_df['Timestamp'].values
gap_ranges = []
if len(miss_mask):
    padded = np.r_[False, miss_mask, False]
    edges = np.flatnonzero(padded[1:] != padded[:-1])
    for a, b in zip(edges[0::2], edges[1::2]):
        bins = b - a
        dur_sec = bins * bin_sec
        if dur_sec >= 120:
            st = pd.Timestamp(ts[a])
            en = pd.Timestamp(ts[b - 1]) +
pd.Timedelta(seconds=bin_sec)
            gap_ranges.append((st, en))
# ----- 그래프 -----
fig, ax1 = plt.subplots(figsize=(15, 6))
# 누락구간 음영
for st, en in gap_ranges:
    if st < end_time and en > start_time:
        ax1.axvspan(max(st, start_time), min(en, end_time),
color='skyblue', alpha=0.35, zorder=0)
# FBG 센서 온도
for col in temp_cols:
    ax1.plot(win_fbg['Timestamp'], win_fbg[col], linewidth=1,
label=col)
# IoT Temperature (옵션)
if show_iot_temp:
    ax1.plot(win_iot['datetime'], win_iot['Temperature'], linewidth=1,
label='IoT Temperature', color='blue')
# 포맷
ax1.set_title(title, fontsize=14)
ax1.set_xlabel('Time')
ax1.set_ylabel('Temperature (°C)')
ax1.set_xlim([start_time, end_time])
if y_limits:
    ax1.set_ylim(y_limits)
ax1.yaxis.set_major_formatter(FormatStrFormatter('%.2f'))
ax1.xaxis.set_major_formatter(mdates.DateFormatter('%m-%d %H:%M'))
ax1.xaxis.set_major_locator(mdates.HourLocator(interval=48))
ax1.grid(True)
```

```
# IoT Humidity (옵션, 오른쪽 y축)
if show_iot_humidity:
    ax2 = ax1.twinx()
    ax2.plot(win_iot['datetime'], win_iot['Humidity'], 'y--',
alpha=0.4,
            linewidth=1.0, label='IoT Humidity')
    ax2.set_ylabel('Humidity (%)', fontsize=9)
    ax2.set_ylim([45, 100])
else:
    ax2 = None
# 특정 시각 빨간 점선
if mark_times:
    for mt in mark_times:
        ax1.axvline(mt, color='red', linestyle='--', linewidth=1.5)
# 범례 합치기
lines1, labels1 = ax1.get_legend_handles_labels()
if ax2:
    lines2, labels2 = ax2.get_legend_handles_labels()
    lines = lines1 + lines2
    labels = labels1 + labels2
    if 'IoT Humidity' in labels:
        idx = labels.index('IoT Humidity')
        lines = lines[:idx] + lines[idx+1:] + [lines[idx]]
        labels = labels[:idx] + labels[idx+1:] + ['IoT Humidity']
    else:
        lines, labels = lines1, labels1
ax1.legend(lines, labels, loc='upper right', fontsize=8, ncol=2)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

동작 과정 :

1. IoT 데이터 로드
 - 컬럼명을 표준화 (Temperature, Humidity)
 - datetime 변환
2. FBG 데이터 로드
 - 접두어(sensor_prefix)로 센서 열 추출
 - 필요시 선택 센서만 필터링

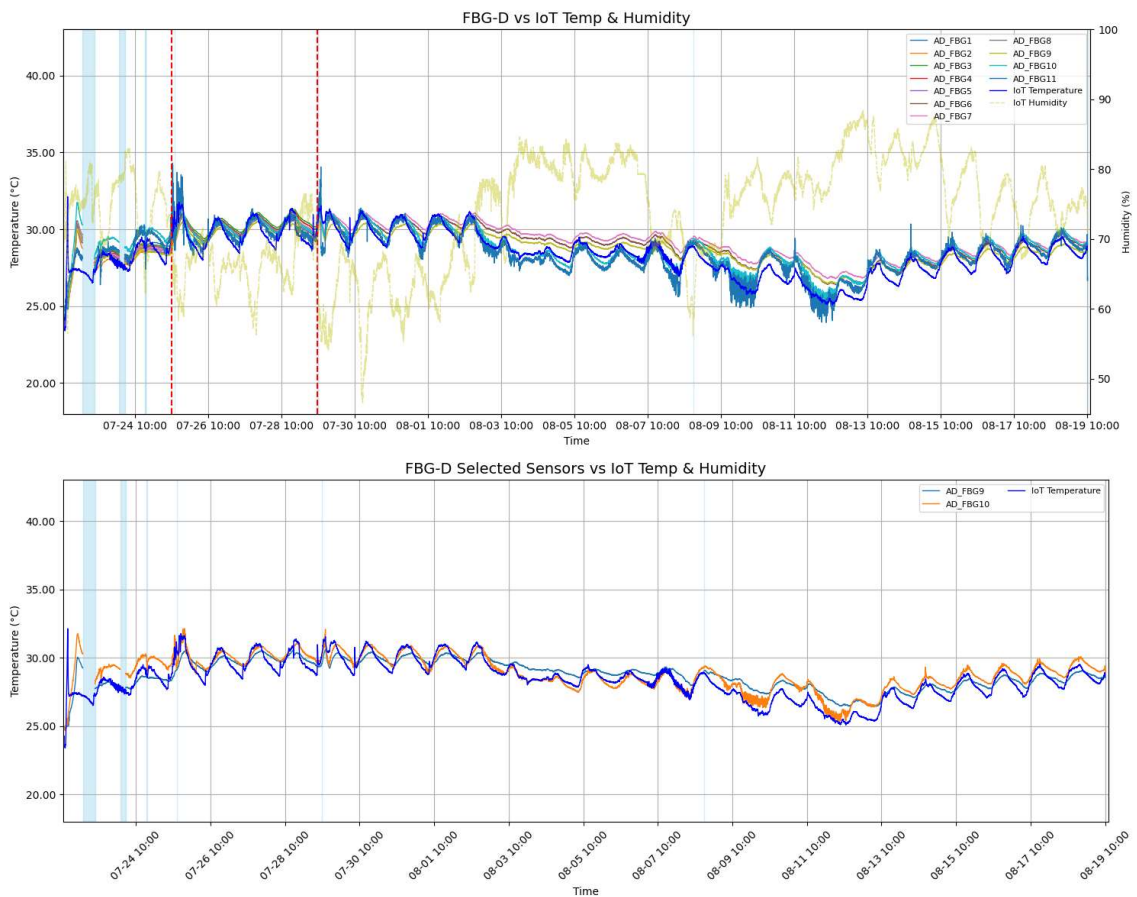
3. 시간 범위 슬라이싱
 - 시작/종료 시각 범위 지정
4. 결측 구간 탐지
 - 모든 센서가 동시에 NaN인 구간을 찾아 음영 표시
5. FBG 온도 플롯
 - 센서별 곡선
6. IoT 온도 플롯 (옵션)
 - show_iot_temp=True일 때만 표시
7. IoT 습도 플롯 (옵션, 오른쪽 y축)
 - show_iot_humidity=True일 때만 표시
8. 특정 이벤트 시각 표시
 - mark_times에 빨간 점선 표시
9. 범례 정리
 - IoT Humidity를 항상 맨 뒤로 정렬

활용 예시

```
plot_fbg_vs_iot(  
    iot_csv=r"C:\Users\user\Desktop\IoT.csv",  
  
    fbg_csv=r"C:\Users\user\Desktop\FBG-D-specimen\FBG-D_temp_only_avg_2min_28Days.csv",  
    sensor_prefix="AD_FBG",  
    start_time="2025-07-22 11:00:00",  
    end_time="2025-08-19 12:00:00",  
    y_limits=(18, 43),  
    mark_times=[  
        pd.Timestamp("2025-07-25 10:00:29"),  
        pd.Timestamp("2025-07-29 09:49:38")  
    ],  
    title="FBG-D vs IoT Temp & Humidity",  
    show_iot_temp=True,  
    show_iot_humidity=True  
)  
plot_fbg_vs_iot(  
    iot_csv=r"C:\Users\user\Desktop\IoT.csv",
```

```
fbg_csv=r"C:\Users\user\Desktop\FBG-D-specimen\FBG-D_temp_only_avg_2min_28Days.csv",
    sensor_prefix="AD_FBG",
    selected_sensors=["AD_FBG9","AD_FBG10"],
    y_limits=(18, 43),
    title="FBG-D Selected Sensors vs IoT Temp & Humidity",
    show_iot_temp=True,
    show_iot_humidity=False
)
```

결과



7) correct_fbg_a_temperature_with_strain()

목적 :

- FBG-A 센서의 온도와 스트레인 데이터를 기준 시각 이후 보정
- $\Delta temp$, $\Delta strain$ 파일을 활용해 보정 적용
- 보정된 온도 CSV와 스트레인 CSV를 따로 저장
- 원하는 기간(start_time, end_time)만 자를 수 있음

- 선택 센서만 확인용 그래프 출력

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
from matplotlib.ticker import FormatStrFormatter
# ----- 한글 폰트 -----
plt.rcParams['font.family'] = 'Malgun Gothic'
plt.rcParams['axes.unicode_minus'] = False

def correct_fbg_a_temperature_with_strain(
    base_path: str,
    delta_temp_path: str,
    delta_strain_path: str,
    save_temp_path: str,
    save_strain_path: str,
    cut_ts: str = "2025-07-25 14:00:00",
    start_time: str = None,
    end_time: str = None,
    y_limits: tuple = (15, 45),
    plot_cols: list = None
):
    """
    FBG-A 온도/스트레인 보정 함수 (기간 지정 가능)
    Parameters
    -----
    base_path : str
        원본 평균온도 파일
    delta_temp_path : str
        Δtemp 파일
    delta_strain_path : str
        Δstrain 파일
    save_temp_path : str
        보정된 온도 저장 경로
    save_strain_path : str
        보정된 스트레인 저장 경로
    cut_ts : str
        보정 기준 시각
    start_time : str
```

```
        분석 시작 시각 (없으면 전체 사용)
end_time : str
        분석 종료 시각 (없으면 전체 사용)
y_limits : tuple
        y축 범위 (min, max)
plot_cols : list
        확인용으로 그래프 그릴 센서 리스트
"""

# ===== 1) 로드 =====
df_base = pd.read_csv(base_path,
parse_dates=['Timestamp']).sort_values('Timestamp')
df_delta_temp = pd.read_csv(delta_temp_path,
parse_dates=['Timestamp']).sort_values('Timestamp')
df_delta_strain = pd.read_csv(delta_strain_path,
parse_dates=['Timestamp']).sort_values('Timestamp')
cut_ts = pd.Timestamp(cut_ts)
if start_time: start_time = pd.Timestamp(start_time)
if end_time: end_time = pd.Timestamp(end_time)
# 센서 컬럼 (원본 파일 기준)
temp_cols = [c for c in df_base.columns if c.startswith("C_FBG")]
# Δtemp 컬럼 통일
delta_temp_cols = [c for c in df_delta_temp.columns if c.startswith("Δ
temp_")]
if not delta_temp_cols:
    rename_map = {c: f'Δtemp_{c}' for c in df_delta_temp.columns if
c.startswith("C_FBG")}
    df_delta_temp = df_delta_temp.rename(columns=rename_map)
    delta_temp_cols = list(rename_map.values())
# Δstrain 컬럼 통일
delta_strain_cols = [c for c in df_delta_strain.columns if
c.startswith("Δε_")]
if not delta_strain_cols:
    rename_map = {c: f'Δε_{c}' for c in df_delta_strain.columns if
c.startswith("C_FBG")}
    df_delta_strain = df_delta_strain.rename(columns=rename_map)
    delta_strain_cols = list(rename_map.values())
# ===== 2) 병합 =====
merged = df_base.copy()
merged = pd.merge(merged, df_delta_temp[['Timestamp']] +
```

```
delta_temp_cols], on='Timestamp', how='left')
    merged = pd.merge(merged, df_delta_strain[['Timestamp'] +
delta_strain_cols], on='Timestamp', how='left')
    # ===== 3) 보정 =====
    row_cut = df_base.loc[df_base['Timestamp'] == cut_ts]
    if row_cut.empty:
        raise ValueError(f"원본 파일에 기준 시각 {cut_ts} 데이터가 없습니다.")
    base_at_cut = row_cut.iloc[0][temp_cols]
    for c in temp_cols:
        dtemp_col = f'Δtemp_{c}'
        dstrain_col = f'Δε_{c}'
        if dtemp_col not in merged.columns:
            merged[dtemp_col] = np.nan
        if dstrain_col not in merged.columns:
            merged[dstrain_col] = np.nan
        # 온도 보정
        merged[c] = np.where(
            merged['Timestamp'] & cut_ts,
            merged[c],
            base_at_cut[c] + merged[dtemp_col]
        )
    # ===== 4) 구간 자르기 =====
    if start_time and end_time:
        merged = merged[(merged['Timestamp'] >= start_time) &
(merged['Timestamp'] < end_time)]
    # ===== 5) 결과 분리 & 저장 =====
    df_temp = merged[['Timestamp'] + temp_cols].copy()
    df_strain = merged[['Timestamp'] + [f'Δε_{c}' for c in
temp_cols]].copy()
    df_temp.to_csv(save_temp_path, index=False, encoding='utf-8-sig')
    print(f"[SAVE] 보정 온도 저장 → {save_temp_path}")
    df_strain.to_csv(save_strain_path, index=False, encoding='utf-8-sig')
    print(f"[SAVE] 보정 스트레인 저장 → {save_strain_path}")
    # ===== 6) 확인용 플롯 =====
    if plot_cols:
        for c in plot_cols:
            if c not in temp_cols:
                continue
```

```
fig, ax = plt.subplots(figsize=(12, 4))
ax.plot(df_base['Timestamp'], df_base[c], label=f'원본 {c}',
linewidth=1.2)
ax.plot(df_temp['Timestamp'], df_temp[c], label=f'보정 {c}',
linewidth=1.8)
ax.axvline(cut_ts, color='k', linestyle='--', linewidth=1,
alpha=0.7, label=f'cut 시각 {cut_ts:%m-%d %H:%M}')
ax.set_title(f"{c}: 원본 vs 보정")
ax.set_xlabel("Time"); ax.set_ylabel("Temperature (°C)")
ax.grid(True); ax.legend(loc='upper right')
ax.xaxis.set_major_formatter(mdates.DateFormatter('%m-%d %H:%M'))

ax.xaxis.set_major_locator(mdates.HourLocator(interval=24))
ax.yaxis.set_major_formatter(FormatStrFormatter('%.2f'))
if y_limits:
    ax.set_ylim(y_limits)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

return df_temp, df_strain
```

동작 과정 :

1. 파일 로드 :

- 원본 (base_path)
- Δtemp (delta_temp_path)
- Δstrain (delta_strain_path)

2. Δtemp / Δstrain 컬럼명 정리

- 만약 접두어($\Delta\text{temp}_$, $\Delta\epsilon_$) 없으면 자동으로 붙임

3. 병합

- Timestamp 기준으로 3개 데이터프레임 합치기

4. 보정 적용

- 기준 시각(cut_ts)의 원본 온도값을 기준으로 이후 시점은
“보정 온도 = cut 시각 온도 + Δtemp ”
- 스트레인 $\Delta\epsilon$ 그대로 사용

5. 기간 자르기

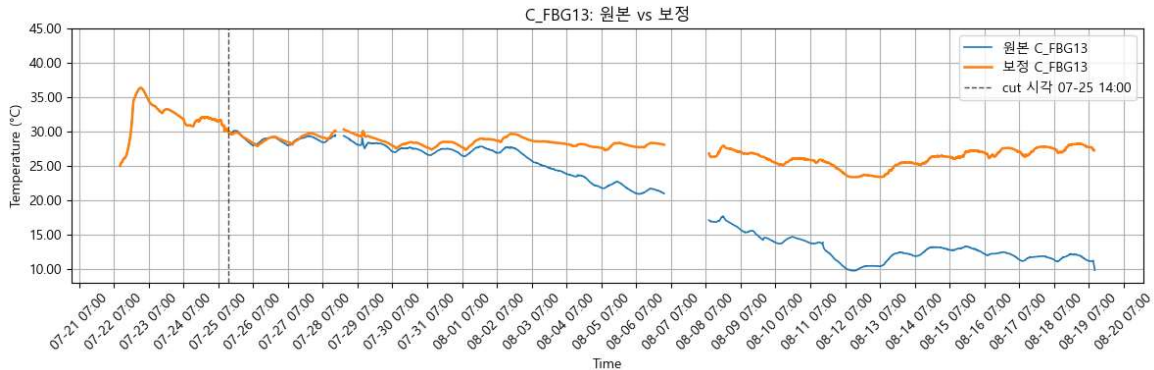
- start_time ~ end_time 지정 가능

6. 결과 저장

- 온도(df_temp) → save_temp_path
- 스트레인(df_strain) → save_strain_path

7. 그래프 확인

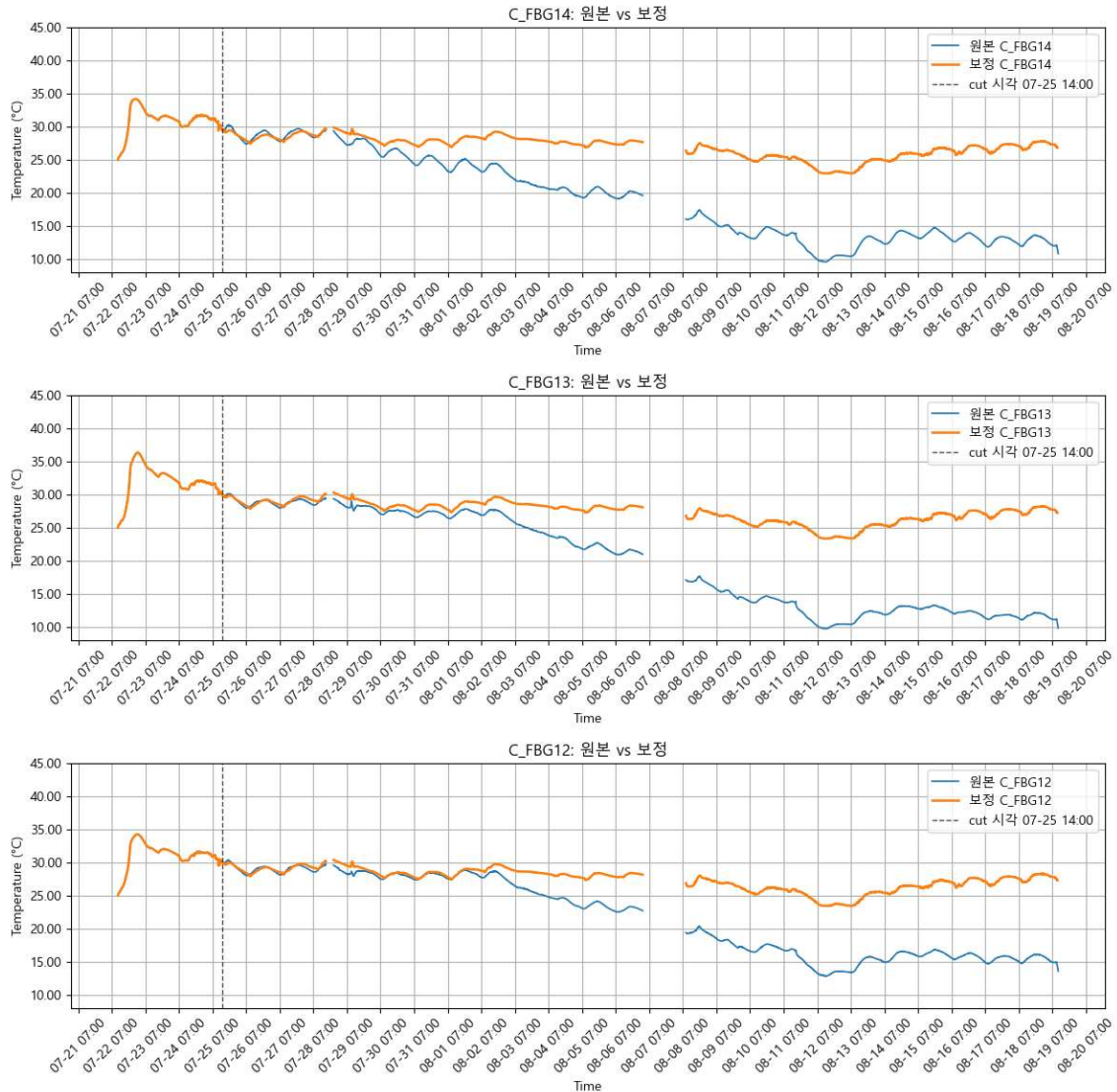
- plot_cols 지정 시 원본 vs 보정 비교 플롯



활용 예시

```
df_temp, df_strain = correct_fbg_a_temperature_with_strain(
    base_path=r"C:\Users\user\Desktop\FBG-A-mockup\FBG-A_temp_only_avg_2min_28Days.csv",
    delta_temp_path=r"C:\Users\user\Desktop\FBG-A-mockup\FBG-A_delta_temp_after_2025-07-25-14.csv",
    delta_strain_path=r"C:\Users\user\Desktop\FBG-A-mockup\FBG-A_delta_strain_after_2025-07-25-14.csv",
    save_temp_path=r"C:\Users\user\Desktop\FBG-A-mockup\FBG-A_temp_corrected.csv",
    save_strain_path=r"C:\Users\user\Desktop\FBG-A-mockup\FBG-A_strain_corrected.csv",
    cut_ts="2025-07-25 14:00:00",
    start_time="2025-07-22 11:00:00",
    end_time="2025-08-19 12:00:00",
    y_limits=(8, 45),
    plot_cols=["C_FBG6", "C_FBG7", "C_FBG8", "C_FBG9", "C_FBG10", "C_FBG11", "C_FBG12", "C_FBG13", 'C_FBG14', 'C_FBG15']
)
```

결과



7) fill_fbg_gaps()

목적 :

- FBG-A 보정 데이터(_corrected)에서 발생한 결측 구간을 자동으로 보완하기 위해 작성된 함수임.
- 결측된 구간을 그대로 두면 이후 “ Δ Temp, Δ Strain 해석 과정에서 불연속성 (discontinuity)”이 생겨 분석 정확도가 떨어짐.
- 따라서, 참조 센서(FBG-B)와의 매핑 규칙을 활용해 A의 결측값을 채워 넣고, 저장 시에는 _corrected 접미어를 제거하여 원래 센서명(C_FBG6 등)으로 저장하도록 함.

```
import pandas as pd
import numpy as np
def fill_fbg_gaps(
    file_a: str,
    file_b: str,
    mapping: dict,
    out_file: str,
    freq: str = "2T"
):
    """
    FBG-A 보정 데이터의 결측 구간을
    참조 FBG-B 데이터를 이용해 보간하여 채움
    Parameters
    -----
    file_a : str
        결측 구간이 있는 보정된 FBG-A CSV 경로
    file_b : str
        참조할 FBG-B CSV 경로
    mapping : dict
        A 센서 → B 센서 매핑 규칙
        (예: {'C_FBG6': 'C_FBG5', ...})      corrected 제거 버전
    out_file : str
        채워넣은 CSV 저장 경로
    freq : str
        데이터 간격 (기본 2분: "2T")
    """
    # ===== CSV 로드 =====
    df_a = pd.read_csv(file_a, parse_dates=['Timestamp'])
    df_b = pd.read_csv(file_b, parse_dates=['Timestamp'])
    # df_a에서 corrected 붙은 컬럼 찾기 → 매핑 키와 매칭
    corrected_cols = {f"{col}_corrected": col for col in mapping.keys() if
f"{col}_corrected" in df_a.columns}
    if not corrected_cols:
        raise ValueError("△ df_a에 '_corrected' 컬럼이 없음. 파일 확인 필
요.")
    # ===== 결측 구간 탐색 =====
    nan_mask = df_a[corrected_cols.keys()].isna().all(axis=1)
    gap_ranges, in_gap = [], False
```

```
for t, is_nan in zip(df_a['Timestamp'], nan_mask):
    if is_nan and not in_gap:
        gap_start = t
        in_gap = True
    elif not is_nan and in_gap:
        gap_end = t
        gap_ranges.append((gap_start, gap_end))
        in_gap = False
    if in_gap:
        gap_ranges.append((gap_start, df_a['Timestamp'].iloc[-1]))
print("□ 찾은 결측 구간:", gap_ranges)
# ===== 각 구간 채우기 =====
for gap_start, gap_end in gap_ranges:
    mask_gap = (df_a['Timestamp'] >= gap_start) & (df_a['Timestamp']
< gap_end)
    for a_col, b_col in mapping.items():
        a_corr_col = f"{a_col}_corrected" # 실제 데이터는 corrected 붙
어있음
        if a_corr_col not in df_a.columns or b_col not in df_b.columns:
            print(f"△ {a_corr_col} 또는 {b_col} 없음 → 스킵")
            continue
        ref_time_start = gap_start - pd.Timedelta(freq)
        ref_time_end = gap_end + pd.Timedelta(freq)
        if ref_time_start not in df_a['Timestamp'].values or
ref_time_end not in df_a['Timestamp'].values:
            print(f"△ {ref_time_start} 또는 {ref_time_end} 없음 → 스킵
{a_corr_col}")
            continue
        # 시작/끝 offset
        ref_a_start = df_a.loc[df_a['Timestamp'] == ref_time_start,
a_corr_col].values[0]
        ref_b_start = df_b.loc[df_b['Timestamp'] == ref_time_start,
b_col].values[0]
        offset_start = ref_a_start - ref_b_start
        ref_a_end = df_a.loc[df_a['Timestamp'] == ref_time_end,
a_corr_col].values[0]
        ref_b_end = df_b.loc[df_b['Timestamp'] == ref_time_end,
b_col].values[0]
        offset_end = ref_a_end - ref_b_end
```

```

times = df_a.loc[mask_gap, 'Timestamp']
if len(times) == 0:
    continue
w = np.linspace(0, 1, len(times))
offsets = offset_start * (1 - w) + offset_end * w
b_vals = df_b.loc[df_b['Timestamp'].isin(times), b_col].values
if len(b_vals) != len(offsets):
    print(f"△ 길이 불일치 {a_corr_col} ({len(b_vals)} vs {len(offsets)}) → 스킵")
    continue
df_a.loc[mask_gap, a_corr_col] = b_vals + offsets
# ===== 저장 (corrected 제거하고 저장) =====
df_out = df_a.copy()
rename_map = {f"{col}_corrected": col for col in mapping.keys()}
df_out = df_out.rename(columns=rename_map)
df_out.to_csv(out_file, index=False)
print("  저장 완료:", out_file)
return df_out

```

처리 절차 :

1. 데이터 로드
 - file_a(FBG-A corrected), file_b(FBG-B temp) CSV를 불러옴.
 - file_a의 센서명은 _corrected가 붙어 있으므로, 이를 인식 후 매핑 대상 컬럼으로 사용.
2. 결측 구간 탐지
 - df_a의 모든 센서값이 NaN인 연속 구간을 찾아 gap_ranges로 정의.
 - 각 구간은 시작 시각(gap_start)과 끝 시각(gap_end)으로 표현됨.
3. 참조 기반 보간(Filling)
 - 구간별로 gap_start - 2분, gap_end + 2분 지점의 A, B 값을 비교하여 offset을 계산.
 - 시작 offset과 끝 offset을 선형 보간(linear interpolation)하여 결측 구간 전체에 적용.
 - 최종적으로 B 값 + offset을 계산하여 A의 결측값을 채움.
4. 컬럼명 정리
 - _corrected 접미어가 붙어 있던 A 센서 컬럼명을 제거하여 “원래 이름(C_FBG6 등)”으로 저장.
 - 즉, 최종 결과는 분석용 표준 센서명과 동일하게 맞춰짐.
5. 결과 저장
 - 보완된 데이터를 CSV(out_file)로 저장.
 - 저장 시, 원본 타임스탬프는 유지되고, 센서명은 _corrected 제거 후 반영됨.

활용 예시

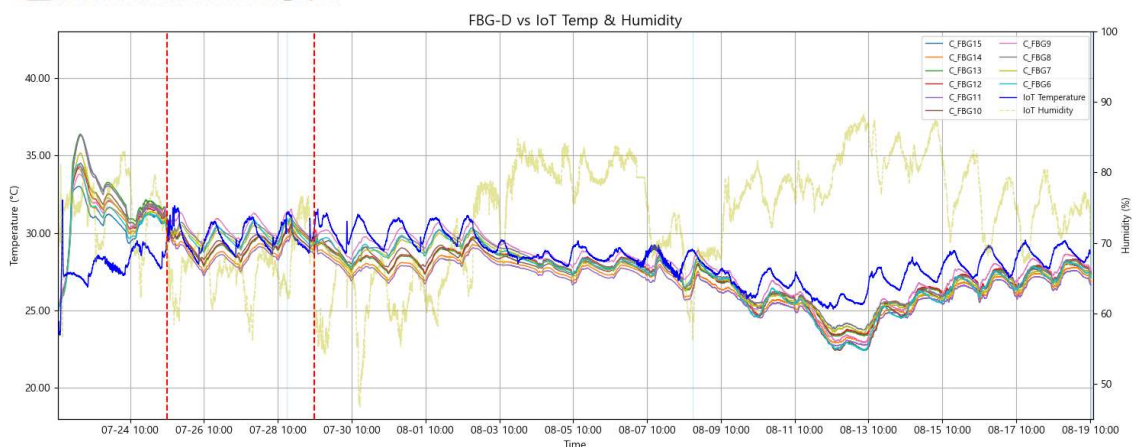
```
mapping_A_B = {
    'C_FBG6': 'C_FBG5',
    'C_FBG7': 'C_FBG5',
    'C_FBG8': 'C_FBG5',
    'C_FBG9': 'C_FBG5',
    'C_FBG10': 'C_FBG4',
    'C_FBG11': 'C_FBG4',
    'C_FBG12': 'C_FBG4',
    'C_FBG13': 'C_FBG4',
    'C_FBG14': 'C_FBG3',
    'C_FBG15': 'C_FBG3'
}

df_filled = fill_fbg_gaps(
    file_a=r"C:\Users\user\Desktop\FBG-A-mockup\FBG-A_corrected_temp.csv",
    file_b=r"C:\Users\user\Desktop\FBG-B-mockup-strain\FBG-B_temp_only_avg_2min_28Days.csv",
    mapping=mapping_A_B,
    out_file=r"C:\Users\user\Desktop\FBG-A-mockup\FBG-A_corrected_all_gaps.csv",
    freq="2T"
)
```

결과



FBG-A_corrected_all_gaps



plot_fbg_vs_iot 함수를 이용해 확인한 결과임.